

R4DS 13 – Functions

Vector Functions

Why Write Functions?

Spot the copy-paste bug:

```
df |> mutate(  
  a = (a - min(a, na.rm = TRUE)) /  
    (max(a, na.rm = TRUE) - min(a, na.rm = TRUE)),  
  b = (b - min(a, na.rm = TRUE)) /    # bug: a!  
    (max(b, na.rm = TRUE) - min(b, na.rm = TRUE)),  
  c = (c - min(c, na.rm = TRUE)) /  
    (max(c, na.rm = TRUE) - min(c, na.rm = TRUE)))
```

Rule of thumb: write a function when you've copied code more than twice.

Three useful types: **vector** (for mutate/filter), **data frame**, and **plot** functions.

Writing a Function

Template: `name <- function(arguments) { body }`

Example — rescale to $[0, 1]$:

```
rescale01 <- function(x) {  
  rng <- range(x, na.rm = TRUE)  
  (x - rng[1]) / (rng[2] - rng[1])  
}  
rescale01(c(-10, 0, 10))
```

```
#> [1] 0.0 0.5 1.0
```

```
rescale01(c(1, 2, 3, NA, 5))
```

```
#> [1] 0.00 0.25 0.50 NA 1.00
```

Now the repetitive code becomes clean:

```
df |> mutate(a = rescale01(a), b = rescale01(b),  
             c = rescale01(c), d = rescale01(d))
```

Improving: Handle Edge Cases

Infinite values cause problems:

```
rescale01(c(1:10, Inf))
```

```
#> [1] 0 0 0 0 0 0 0 0 0 0 NaN
```

Fix: add `finite = TRUE` to `range()`:

```
rescale01 <- function(x) {  
  rng <- range(x, na.rm = TRUE, finite = TRUE)  
  (x - rng[1]) / (rng[2] - rng[1])  
}
```

```
rescale01(c(1:10, Inf))
```

```
#> [1] 0.0000000 0.1111111 0.2222222 0.3333333 0.4444444
```

```
#> [6] 0.5555556 0.6666667 0.7777778 0.8888889 1.0000000
```

```
#> [11] Inf
```

Strategy: get the basics right, then handle edge cases iteratively.

Mutate Functions

Return same-length output — use in `mutate()` and `filter()`:

```
clamp <- function(x, min, max) {  
  case_when(x < min ~ min, x > max ~ max,  
            .default = x)  
}  
clamp(1:10, min = 3, max = 7)
```

```
#> [1] 3 3 3 4 5 6 7 7 7 7
```

```
first_upper <- function(x) {  
  str_sub(x, 1, 1) <-  
    str_to_upper(str_sub(x, 1, 1))  
  x  
}  
first_upper("hello")
```

```
#> [1] "Hello"
```

Other useful patterns: `z_score()`, `clean_number()`, `fix_na()`.

Summary Functions

Return a single value — use in `summarize()`:

```
commas <- function(x) {  
  str_flatten(x, collapse = ", ",  
              last = " and ")  
}  
commas(c("cat", "dog", "pigeon"))
```

```
#> [1] "cat, dog and pigeon"
```

Pass `na.rm` through to the underlying function:

```
cv <- function(x, na.rm = FALSE) {  
  sd(x, na.rm = na.rm) / mean(x, na.rm = na.rm)  
}  
  
n_missing <- function(x) sum(is.na(x))  
  
mape <- function(actual, predicted) {  
  sum(abs((actual - predicted) / actual)) /  
    length(actual)  
}
```

Exercises (Vector Functions)

- 1 Turn these into functions. What would you name each?

```
mean(is.na(x))           # snippet 1
x / sum(x, na.rm = TRUE) # snippet 2
round(x / sum(x, na.rm = TRUE) * 100, 1)
```

- 2 Rewrite `rescale01()` so `-Inf` $\rightarrow$ 0 and `Inf` $\rightarrow$ 1.
- 3 Write a function to compute age in years from a vector of birthdates.
- 4 Write functions for variance and skewness of a numeric vector.
- 5 Write `both_na()`: takes two vectors, returns count of positions with `NA` in both.
- 6 What do these do? Why are they useful despite being short?

```
is_directory <- function(x) file.info(x)$isdir
is_readable  <- function(x) file.access(x, 4) == 0
```


Solutions (Vector Functions) — 1–3

1

```
prop_na <- function(x) mean(is.na(x))
prop_of_total <- function(x)
  x / sum(x, na.rm = TRUE)
pct_of_total <- function(x)
  round(x / sum(x, na.rm = TRUE) * 100, 1)
```

2

```
rescale01 <- function(x) {
  rng <- range(x, na.rm = TRUE, finite = TRUE)
  y <- (x - rng[1]) / (rng[2] - rng[1])
  y[x == -Inf] <- 0; y[x == Inf] <- 1; y
}
rescale01(c(-Inf, 1:5, Inf))
```

```
#> [1] 0.00 0.00 0.25 0.50 0.75 1.00 1.00
```

3

```
age_years <- function(birthdate) {
  as.numeric(difftime(Sys.Date(), birthdate,
    units = "days")) / 365.25
}
```

Solutions (Vector Functions) — 4–6

4

```
variance <- function(x, na.rm = FALSE) {  
  if (na.rm) x <- x[!is.na(x)]  
  n <- length(x)  
  sum((x - mean(x))^2) / (n - 1)  
}  
  
skewness <- function(x, na.rm = FALSE) {  
  if (na.rm) x <- x[!is.na(x)]  
  n <- length(x); m <- mean(x); s <- sd(x)  
  (n / ((n-1) * (n-2))) * sum(((x-m) / s)^3)  
}
```

5

```
both_na <- function(x, y) {  
  sum(is.na(x) & is.na(y))  
}  
  
both_na(c(1, NA, 3, NA), c(NA, NA, 3, NA))
```

```
#> [1] 2
```

- 6 They wrap low-level system calls into readable names. Short functions are valuable when they **improve readability**.

Data Frame Functions

The Indirection Problem

This fails — dplyr looks for a column literally named `group_var`:

```
grouped_mean <- function(df, group_var, mean_var) {  
  df |>  
    group_by(group_var) |>  
    summarize(mean(mean_var))  
}  
  
diamonds |> grouped_mean(cut, carat)  
#> Error: column `group_var` not found
```

The issue: dplyr uses **tidy evaluation** — it looks for column names in the data, not in R's environment.

Embracing with { }

Solution: wrap arguments in `{{ }}` (“embrace”) to tell dplyr to look inside:

```
grouped_mean <- function(df, group_var, mean_var) {  
  df |>  
    group_by({{ group_var }}) |>  
    summarize(mean({{ mean_var }}))  
}  
diamonds |> grouped_mean(cut, carat)
```

```
#> # A tibble: 5 x 2  
#>   cut      `mean(carat)`  
#>   <ord>          <dbl>  
#> 1 Fair           1.05  
#> 2 Good           0.849  
#> 3 Very Good     0.806  
#> 4 Premium       0.892  
#> ... [output truncated]
```

Rule: if an argument is passed to a dplyr verb, embrace it with `{{ }}`.

Common Use Cases

Flexible summary function:

```
summary6 <- function(data, var) {  
  data |> summarize(  
    min = min({{ var }}, na.rm = TRUE),  
    mean = mean({{ var }}, na.rm = TRUE),  
    median = median({{ var }}, na.rm = TRUE),  
    max = max({{ var }}, na.rm = TRUE),  
    n = n(), n_miss = sum(is.na({{ var }}}),  
    .groups = "drop")  
}  
diamonds |> summary6(carat)
```

```
#> # A tibble: 1 x 6  
#>   min mean median max      n n_miss  
#>   <dbl> <dbl> <dbl> <dbl> <int> <int>  
#> 1   0.2 0.798   0.7  5.01 53940     0
```

Works on grouped data and with expressions:

```
diamonds |> group_by(cut) |> summary6(carat)  
diamonds |> group_by(cut) |> summary6(log10(carat))
```

More Use Cases

Count with proportions:

```
count_prop <- function(df, var, sort = FALSE) {  
  df |>  
    count({{ var }}, sort = sort) |>  
    mutate(prop = n / sum(n))  
}  
diamonds |> count_prop(clarity)
```

```
#> # A tibble: 8 x 3  
#>   clarity      n  prop  
#>   <ord>    <int> <dbl>  
#> 1 I1      741 0.0137  
#> 2 SI2     9194 0.170  
#> ... [output truncated]
```

Unique values under a condition:

```
unique_where <- function(df, condition, var) {  
  df |> filter({{ condition }}) |>  
    distinct({{ var }}) |> arrange({{ var }})  
}  
flights |> unique_where(month == 12, dest)
```

Data-Masking vs. Tidy-Selection

Two types of tidy evaluation:

- **Data-masking** (`filter()`, `mutate()`, `summarize()`): works with expressions like `x + 1`
- **Tidy-selection** (`select()`, `rename()`, `across()`): works with selections like `starts_with("x")`

Problem: `group_by()` is data-masking, but you want to pass multiple variables. Use `pick()`:

```
count_missing <- function(df, group_vars, x_var) {  
  df |>  
    group_by(pick({{ group_vars }})) |>  
    summarize(n_miss = sum(is.na({{ x_var }})),  
              .groups = "drop")  
}  
flights |>  
  count_missing(c(year, month, day), dep_time)
```

```
#> # A tibble: 365 x 4  
#>   year month   day n_miss  
#>   <int> <int> <int>   <int>  
#> 1  2013     1     1     4  
#> 2  2013     1     2     8  
#> ... [output truncated]
```


Exercises (Data Frame Functions)

- 1 Using `nycflights13`, write functions that:
 - a Find cancelled or >1 hour delayed flights
 - b Count cancelled and >1 hour delayed by group
 - c Like (a) but with user-supplied hours
 - d Summarize weather: min, mean, max of any variable
 - e Convert clock time to decimal hours
- 2 Which arguments use tidy evaluation in: `distinct()`, `count()`, `group_by()`, `rename_with()`, `slice_min()`, `slice_sample()`?
- 3 Generalize `count_prop()` to accept any number of variables.

```
filter_severe <- function(df, hours = 1) {  
  df |> filter(is.na(arr_time) |  
    dep_delay > hours * 60)  
}  
  
summarize_severe <- function(df) {  
  df |> summarize(  
    n_cancelled = sum(is.na(arr_time)),  
    n_delayed = sum(dep_delay > 60, na.rm = TRUE))  
}
```

`hours` parameter in `filter_severe()` handles both 1a and 1c.

```
summarize_weather <- function(df, var) {  
  df |> summarize(  
    min = min({{ var }}, na.rm = TRUE),  
    mean = mean({{ var }}, na.rm = TRUE),  
    max = max({{ var }}, na.rm = TRUE))  
}  
  
standardize_time <- function(df, var) {  
  df |> mutate({{ var }} :=  
    ({{ var }} %% 100) + ({{ var }} %% 100) / 60)  
}
```

Note: `standardize_time()` uses the walrus operator `:=` to modify the column in place.

- ② **Data-masking:** `distinct(...)`, `count(...)`, `group_by(...)`, `slice_min(order_by)`. **Tidy-selection:** `rename_with(.cols)`. `slice_sample()` uses neither for column selection.
- ③ Use `...` instead of a single `var`:

```
count_prop <- function(df, ..., sort = FALSE) {
  df |>
    count(..., sort = sort) |>
    mutate(prop = n / sum(n))
}
diamonds |> count_prop(clarity, cut)
```

```
#> # A tibble: 40 x 4
#>   clarity cut          n    prop
#>   <ord>   <ord>    <int>  <dbl>
#> 1 I1     Fair     210 0.00389
#> 2 I1     Good      96 0.00178
#> 3 I1     Very Good  84 0.00156
#> ... [output truncated]
```

Plot Functions

Building Plot Functions

Same `{{ }}` pattern — embracing works inside `aes()`:

```
histogram <- function(df, var, binwidth = NULL) {  
  df |>  
    ggplot(aes(x = {{ var }})) +  
    geom_histogram(binwidth = binwidth)  
}
```

```
diamonds |> histogram(carat, 0.1)
```

Can still add layers:

```
diamonds |> histogram(carat, 0.1) +  
  labs(x = "Size (in carats)")
```

More Variables and Labels

```
linearity_check <- function(df, x, y) {  
  df |> ggplot(aes(x = {{ x }}, y = {{ y }})) +  
    geom_point() +  
    geom_smooth(method = "loess", formula = y ~ x,  
      color = "red", se = FALSE) +  
    geom_smooth(method = "lm", formula = y ~ x,  
      color = "blue", se = FALSE)  
}  
starwars |> filter(mass < 1000) |>  
  linearity_check(mass, height)
```

Dynamic labels with `rlang::engluce()` — `{{var}}` inserts the variable name:

```
histogram <- function(df, var, binwidth) {  
  label <- rlang::engluce(  
    "A histogram of {{var}} with binwidth {binwidth}")  
  df |> ggplot(aes(x = {{ var }})) +  
    geom_histogram(binwidth = binwidth) +  
    labs(title = label)  
}
```

Combining with Tidyverse

Use the **walrus operator** `:=` to create variable names with `{{ var }}` on the left side:

```
sortedBars <- function(df, var) {  
  df |>  
    mutate({{ var }} :=  
      fct_rev(fct_infreq({{ var }}})) |>  
    ggplot(aes(y = {{ var }})) +  
    geom_bar()  
}  
diamonds |> sortedBars(clarity)
```

`:=` works like `=` but allows embracing on the **left** side of assignment.

Exercises (Plot Functions)

Build up a rich plotting function incrementally:

- 1 Draw a scatterplot given a dataset and x and y variables.
- 2 Add a line of best fit (linear model, no standard errors).
- 3 Add a title that includes the variable names.

Solutions (Plot Functions)

Build incrementally — final version includes all three:

```
# 1. Start: scatterplot
scatter <- function(df, x, y) {
  df |> ggplot(aes(x = {{ x }}, y = {{ y }})) +
    geom_point()
}

# 2+3. Add line of best fit + dynamic title
scatter <- function(df, x, y) {
  df |> ggplot(aes(x = {{ x }}, y = {{ y }})) +
    geom_point() +
    geom_smooth(method = "lm", formula = y ~ x,
      se = FALSE) +
    labs(title = rlang::englue("{{x}} vs {{y}}"))
}
```

Style

Function Naming and Formatting

Names: use verbs, be descriptive:

# Too vague	# Better
<code>f()</code>	<code>impute_missing()</code>
<code>my_function()</code>	<code>collapse_years()</code>

Formatting: extra indent inside function body:

```
# Good --- body indented extra 2 spaces
density <- function(color, facets,
                    binwidth = 0.1) {
  diamonds |>
    ggplot(aes(x = carat,
              y = after_stat(density),
              color = {{ color }})) +
    geom_freqpoly(binwidth = binwidth) +
    facet_wrap(vars({{ facets }}))
}
```

Exercises (Style)

- 1 What do these do? Suggest better names:

```
f1 <- function(string, prefix) {  
  str_sub(string, 1, str_length(prefix)) == prefix  
}  
  
f3 <- function(x, y) {  
  rep(y, length.out = length(x))  
}
```

- 2 Make a case for `norm_r()`, `norm_d()` over `rnorm()`, `dnorm()`. Then make the opposite case.

- ① `f1` checks if a string starts with a given prefix → `has_prefix()` or `str_has_prefix()`. `f3` repeats `y` to match the length of `x` → `match_length()` or `recycle()`.
- ② For `norm_r()`: common prefix groups all normal functions together — autocomplete shows `norm_...`. Aids discovery. **Against:** R's convention (`rnorm`, `dnorm`, `pnorm`) groups by **operation** across distributions (`r...` = random, `d...` = density). Muscle memory and decades of convention matter.